



**Pontificia Universidad
Católica del Ecuador**
Seréis mis testigos

MANABÍ

ESTUDIANTES

Steven Alexandre Mero Castro

Joshua Mateo Castañeda Real

Kevin Abiud Vera Zambrano

Saul Velez Ponce

Alex Palma Delgado

Carlos Jr Andrade

DOCENTE

Ingeniero José Naranjo

ASIGNATURA

Práctica en gestión de la información

PARALELO

Séptimo Semestre.

Grupo 2



Introducción a Entity Framework con el ecosistema .NET

Desarrollar sistemas que permitan:

- 1. Conexión ADO.NET:** Establecer conexión a SQL Server con SqlConnection.
- 2. Entity Framework:** Implementar CRUD con DbContext y DbSet.
- 3. Migraciones:** Crear y aplicar migraciones para la base de datos.
- 4. Code First:** Definir modelos y relaciones con Data Annotations.
- 5. LINQ:** Utilizar LINQ para consultas complejas.

Joshua Mateo: Conexión a BD desde C#/.NET: SqlConnection, SqlCommand, DataReader.

Steven Mero: Introducción a Entity Framework: DbContext, DbSet, modelos.

Carlos Jr: Migraciones y enfoques: Code First vs Database First vs Model First.

Abiud Vera: CRUD en .NET Core con SQL Server: implementación práctica.

Saul Velez: Entity Framework vs ADO.NET: comparativa y casos de uso.

Alex Palma: Desafío final y ejercicio para la clase.

Conexión a BD desde C#/.NET: SqlConnection, SqlCommand, DataReader

Es el acceso a bases de datos en aplicaciones desarrolladas con C# y .NET se realiza comúnmente mediante ADO.NET, un conjunto de clases que permiten establecer conexiones, ejecutar consultas y manipular información almacenada en sistemas gestores de bases de datos como SQL Server. Entre las clases más importantes de ADO.NET se encuentran SqlConnection, SqlCommand y SqlDataReader, las cuales forman la base de la comunicación entre una aplicación y una base de datos relacional.



- **SqlConnection**

Es una clase que se encarga de establecer y administrar la conexión entre una aplicación y una base de datos SQL Server. Su función principal es proporcionar un canal de comunicación seguro mediante una cadena de conexión que contiene información como el servidor, la base de datos y las credenciales necesarias para el acceso. Sin una conexión activa, la aplicación no puede ejecutar consultas ni interactuar con los datos almacenados.

Además de abrir y cerrar conexiones, (SqlConnection) gestiona los recursos asociados a la comunicación con el servidor de bases de datos. Por esta razón, es recomendable utilizar la instrucción (using), ya que garantiza la liberación automática de recursos una vez finalizada la operación.

- **SqlCommand**

Es una clase que representa una instrucción SQL que será enviada al servidor de bases de datos para su ejecución. Permite ejecutar consultas de selección (SELECT), inserción (INSERT), actualización (UPDATE) y eliminación (DELETE), así como procedimientos almacenados.

Dependiendo del tipo de operación que se desee realizar, SqlCommand proporciona distintos métodos de ejecución.

Entre los más utilizados se encuentran:

ExecuteReader: que devuelve un conjunto de resultados;

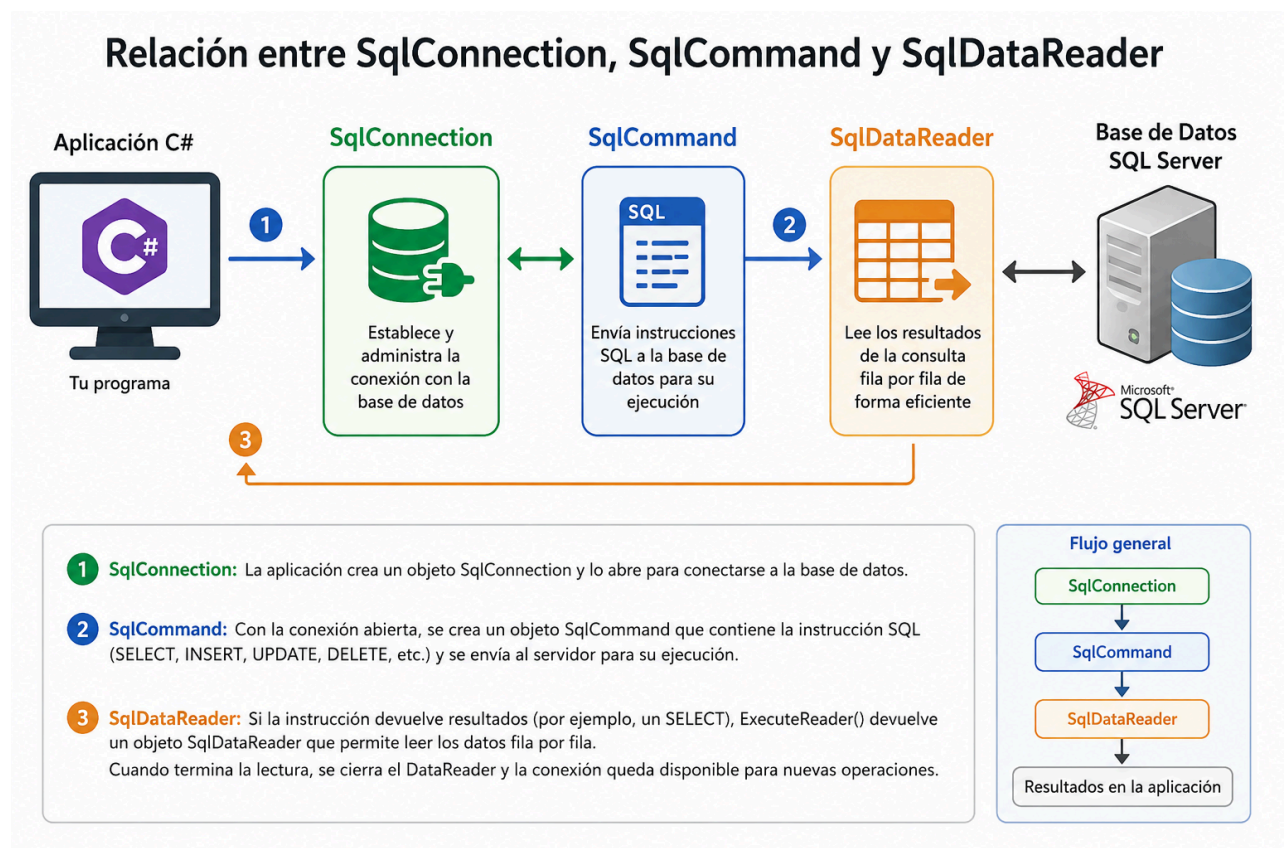
ExecuteNonQuery: empleado para operaciones que modifican datos **ExecuteScalar:** utilizado cuando se espera un único valor como resultado.



- SqlDataReader

Esta clase se utiliza para leer los resultados obtenidos de una consulta SQL de forma secuencial y eficiente. Es especialmente adecuada cuando se necesita recorrer grandes cantidades de registros, ya que mantiene una conexión abierta con la base de datos y recupera los datos fila por fila, reduciendo el consumo de memoria.

El objeto `SqlDataReader` se obtiene mediante el método `ExecuteReader()` de un objeto `SqlCommand`. Posteriormente, el método `Read()` permite avanzar por cada registro devuelto por la consulta y acceder a los valores de las columnas mediante índices o nombres de campo.





Introducción a Entity Framework: DbContext, DbSet y Modelos

Entity Framework Core es un mapeador objeto-relacional (ORM) moderno de código abierto para .NET. Como señala (*Microsoft*, 2026) Su propósito es permitirnos interactuar con una base de datos relacional utilizando objetos de C# fuertemente tipados, eliminando casi por completo la necesidad de escribir código de acceso a datos de bajo nivel como abrir conexiones manualmente, crear comandos SQL intermedios o mapear lectores de datos.

Resuelve la desconexión de la impedancia objetual-relacional. En el desarrollo tradicional con ADO.NET, existe una enorme fricción debido a que las bases de datos organizan la información en tablas jerárquicas/relacionales, mientras que el software la organiza en grafos de objetos con herencia y encapsulamiento. (Anonimo, S.f)

Importancia en el desarrollo de software

- **Aumento drástico de la productividad:** Reduce el código repetitivo
- **Seguridad por defecto:** Las consultas escritas mediante LINQ se parametrizan automáticamente bajo el capó, lo que neutraliza por completo los ataques de SQL Injection.
- **Portabilidad:** Permite cambiar el motor de base de datos por ejemplo, de SQL Server a PostgreSQL modificando únicamente una línea de configuración.

Componentes Principales

- **Modelos:** Son clases simples de C# que representan la estructura de los datos. Cada propiedad de la clase suele corresponder a una columna en la tabla de la base de datos.



- **DbSet:** Es una colección genérica que representa un conjunto de entidades específicas en el contexto. Es la puerta de entrada para realizar operaciones de consulta y guardado para una entidad en particular, conceptualmente equivale a una tabla o vista de la base de datos.
- **DbContext:** Es el componente central más importante, representa una sesión con la base de datos. Se encarga de gestionar las conexiones, configurar los mapeos, rastrear los cambios hechos en los objetos y coordinar las transacciones al salvar los datos.

El **DbContext** actúa como el contenedor principal, dentro de él se declaran los **DbSet** (las tablas accesibles), y cada DbSet opera manipulando instancias de los **Modelos** (las filas individuales).

Ventajas

- Abstracción de alto nivel, no escribes SQL en cadenas de texto propensas a errores tipográficos.
- Mantenimiento ágil gracias a las migraciones automáticas fuertemente tipadas.
- Soporte nativo para consultas asíncronas ideal para optimizar hilos de servidor.

Desventajas

- Sobrecarga de rendimiento (Overhead): El proceso de compilación de consultas LINQ a SQL y la materialización de objetos consume más CPU y memoria.
- Pérdida de control directo: En consultas masivas o reportes sumamente complejos con múltiples subconsultas, el SQL generado de forma automatizada puede no ser el más óptimo.



Migraciones y Enfoques en Entity Framework

¿Qué son las Migraciones?

Las migraciones en Entity Framework permiten actualizar y mantener sincronizada la base de datos con los cambios realizados en las clases del proyecto.

Gracias a las migraciones, cuando se agrega una nueva propiedad, tabla o relación en el código, Entity Framework puede generar automáticamente los scripts SQL necesarios para reflejar esos cambios en la base de datos.

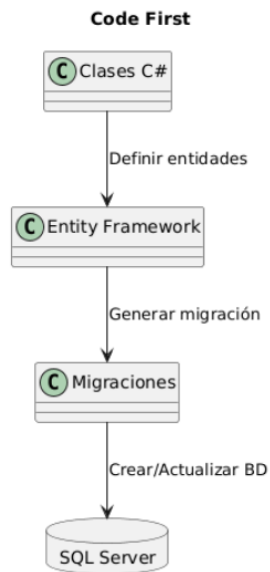
Ventajas

- Control de versiones de la base de datos.
- Actualización automática del esquema.
- Facilita el trabajo en equipo.
- Evita crear manualmente scripts SQL.

Enfoque Code First

En Code First primero se crean las clases en C# y luego Entity Framework genera automáticamente la base de datos.

Flujo



Ventajas

- Desarrollo rápido.
- Mayor control desde el código.
- Fácil uso de migraciones.
- Ideal para proyectos nuevos.

Desventajas

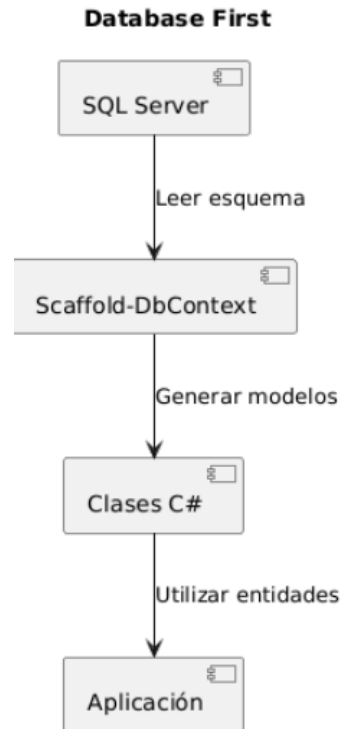
- Puede ser complicado en bases de datos muy grandes.
- Requiere conocimientos de Entity Framework.

Enfoque Database First

Primero se crea la base de datos y luego Entity Framework genera automáticamente las clases.



Flujo



Ventajas

- Ideal para bases de datos ya existentes.
- Aprovecha sistemas heredados.
- Fácil integración con proyectos empresariales.

Desventajas

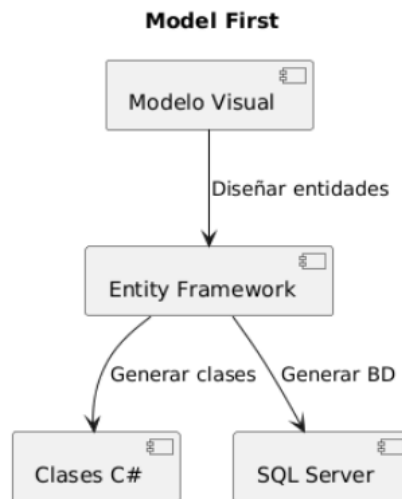
- Menor control sobre el código generado.
- Las modificaciones deben hacerse en la base de datos.

4. Enfoque Model First

Definición

Primero se diseña un modelo gráfico (diagrama entidad-relación) y posteriormente se generan las clases y la base de datos.

Flujo



A partir del modelo, Visual Studio genera:

- Clases
- Relaciones
- Scripts SQL

Ventajas

- Diseño visual intuitivo.
- Fácil comprensión para usuarios no técnicos.
- Buena documentación del sistema.



Desventajas

- Menos utilizado actualmente.
- Menor flexibilidad que Code First.
- No disponible en Entity Framework Core moderno.

Comparación de los Enfoques

Característica	Code First	Database First	Model First
Punto de inicio	Código	Base de datos	Modelo gráfico
Uso de migraciones	Sí	Limitado	Sí
Ideal para proyectos nuevos	Sí	No	Sí
Ideal para BD existente	No	Sí	No
Facilidad de mantenimiento	Alta	Media	Media
Popularidad actual	Muy alta	Alta	Baja

Entity Framework vs ADO.NET. ¿Qué son?

ADO.NET es el acceso a datos de bajo nivel de .NET: envías SQL directamente y manejas conexiones, comandos y lectores manualmente. Entity Framework (EF) es un ORM construido sobre ADO.NET que mapea clases C# a tablas y genera el SQL por ti.



Comparativa directa

Aspecto	ADO.NET	Entity Framework
Abstracción	Baja — SQL explícito	Alta — LINQ → SQL
Curva de aprendizaje	Moderada (requiere saber SQL)	Mayor al inicio, luego más ágil
Rendimiento	Máximo control, mínimo overhead	Ligero overhead; EF Core es muy eficiente
Productividad	Más código boilerplate	Mucho menos código
Consultas complejas	Excelente — SQL nativo	Posible pero puede generar SQL subóptimo
Mantenimiento	Propenso a errores en strings SQL	Refactoring seguro, tipado en compile-time
Migraciones de schema	Manual	Automáticas con dotnet ef migrations
Soporte multi-BD	Dependes del driver	Cambias el provider (SQL Server, Postgres, SQLite...)

Cuándo usar ADO.NET

- **Rendimiento crítico:** queries con millones de filas, ETL, procesamiento en batch
- **SQL muy complejo:** PIVOT, CTEs recursivos, hints de índices, procedimientos almacenados complejos



- **Aplicaciones legadas** con esquemas irregulares difíciles de mapear
- **Control total** sobre la conexión, transacciones y el wire protocol
- **Microservicios ultra-ligeros** donde no quieres el peso de un ORM

Cuándo usar Entity Framework (Core)

- **CRUD estándar** y operaciones de negocio cotidianas
- **Equipos** donde no todos dominan SQL — LINQ es más accesible
- **Iteración rápida:** prototipos, startups, proyectos con schema cambiante
- **DDD / arquitecturas en capas** donde quieres un modelo de dominio rico
- **Testing** — EF Core tiene InMemory provider para tests unitarios sin BD

Regla práctica

Situación	Recomendación
App nueva de negocio	EF Core por defecto
Query lenta en EF	Optimiza con <code>.AsNoTracking()</code> , proyecciones, o raw SQL puntual
Servicio de alta carga con queries simples	Dapper o ADO.NET
Todo lo demás	EF Core + raw SQL donde sea necesario

En proyectos modernos, EF Core cubre el 90% de los casos con mucha menos fricción. ADO.NET sigue siendo indispensable cuando cada milisegundo y cada byte importan.



Referencias bibliográficas

Anónimo. (S.f). 5.- *El desface Objeto-Relacional* | *PROG11 CONTENIDOS*. S'Arreplec.

Retrieved June 23, 2026, from

https://sarreplec.caib.es/pluginfile.php/11712/mod_resource/content/1/5_el_desface_objetorelacional.html

Información general de Entity Framework Core - EF Core. (2026, March 27). Microsoft

Learn. Retrieved June 23, 2026, from <https://learn.microsoft.com/es-es/ef/core/>

Cmastr. (n.d.). *Recuperación de datos mediante dataReader - ADO.NET*. Microsoft Learn.

<https://learn.microsoft.com/es-es/dotnet/framework/data/adonet/retrieving-data-using-a-datareader>

Flyingweseal. (2015, November 24). *C# SQL Connection try/catch vs using vs try/catch w/ using*. Stack Overflow.

<https://stackoverflow.com/questions/33898969/c-sharp-sql-connection-try-catch-vs-using-vs-try-catch-w-using>

Entity Framework Overview - ADO.NET. (2021, September 15). Microsoft Learn. Retrieved

June 30, 2026, from

<https://learn.microsoft.com/en-us/dotnet/framework/data/adonet/ef/overview>

Prajapati, Y. (2025, June 14). *ADO.NET vs Entity Framework vs EF Core — Key Differences*

Explained. Yash Prajapati. Retrieved June 30, 2026, from



**Pontificia Universidad
Católica del Ecuador**
Seréis mis testigos

MANABÍ

<https://ysprajapatit.medium.com/ado-net-vs-entity-framework-vs-ef-core-key-differences-explained-ddf38af5e7ab>

Dotnet-Bot. (n.d.). *SqlCommand Clase (System.Data.SqlClient)*. Microsoft Learn.

<https://learn.microsoft.com/es-es/dotnet/api/system.data.sqlclient.sqlcommand?view=netframework-4.8.1>

Friedman, J., & Friedman, J. (2018, July 2). *Lesson 03: The SqlCommand Object*. C#

Station. <https://csharp-station.com/Tutorial/AdoDotNet/Lesson03>